

Coursework: Performance Modelling, Vectorisation and GPU programming

C. Reed

COMP52315

Submitted: April 21, 2026

I. PERFORMANCE MODELLING

A. 1.1

Function	Loads	Stores	FLOPs	Operational Intensity (FLOP / Bytes)
function_a()	$2N - k$	0	$2N - k + 2$	$\frac{1}{8}$
function_b()	$3N$	N	N	$\frac{1}{32}$
function_c()	$N^2 + N$	N^2	$2N^2 - 2N$	$\frac{N-1}{8N+4}$ ($\frac{1}{8}$ for large N)
function_d()	$\frac{1}{2}(N^2 + 5N)$	$N + \frac{N^2}{2}$	$\frac{3N^2}{2}$	$\frac{3N}{16N+56}$ ($\frac{3}{16}$ for large N)

TABLE I: Summary of data accesses, floating-point operations, and operational intensity for each function.

For all functions I have considered the best case operational intensity to be that which has the highest flops executed per bytes transferred dependent on the branch. I have also assumed a perfect memory model in which data is moved to cache only once. Hence for the number of loads I have counted unique memory access rather than total memory accesses. It is worth noting I have considered a write operation as 1 load and 1 store, unless the memory location had previously been loaded into cache (as per the perfect memory model). I have also assumed that every single write operation contributes a single store, even though in reality many of these stores are unnecessary and a compiler will optimise these out (this will be relevant for function c and d).

For function A, there are two divergent branches and thus the exact number of loads, stores and FLOPs is unknown until runtime. I have assumed that out of N iterations of the function loop, there are k iterations where $S < 10$ and $N - k$ iterations where $S \geq 10$. The variable s is treated as a scalar kept in a register therefore accessing this does not contribute a load or a store. The *sqr*t and division after the loop provide a constant two FLOPS which I have not counted towards the operational Intensity. Both branches have the same operational intensity thus the best case is irrespective of which branches are chosen at runtime.

Function B is a simple vector addition with the number of loads, stores and FLOPs being trivial.

For function C, under the perfect cache model I have assumed that $x[i]$ and $y[i]$ are loaded only once each, with both contributing N loads. The inner loop executes $N(N - 1)$ times which implies loading

$A[i*N+j]$ contributes $N(N-1)$ loads. In counting the stores, $y[i]$ is initially stored N times for the initialisation and then a further $N(N-1)$ times during the inner loop. As mentioned in the preamble in reality these inner loop stores would be optimised away by a compiler and only 1 store per outer loop would actually occur. For every inner loop there are 2 FLOPs.

Similarly for function D, I have assumed $w[i]$ and $u[i]$ are loaded only once, each contributing N loads. I have assumed that N is even and so the inner loop executes $\frac{N}{2}$ times. $x[j]$ only has to be loaded on the first pass of the inner loop contributing $\frac{N}{2}$ loads and finally $A[i*N+j]$ is loaded on every inner loop contributing $\frac{N^2}{2}$ loads. Similarly to function c, the outer loop contributes N stores for $w[i]$ then $\frac{N^2}{2}$ stores in the inner loop. Although many of these would inner loop stores are unnecessary. For every inner loop there are 3 FLOPs.

B. 1.2

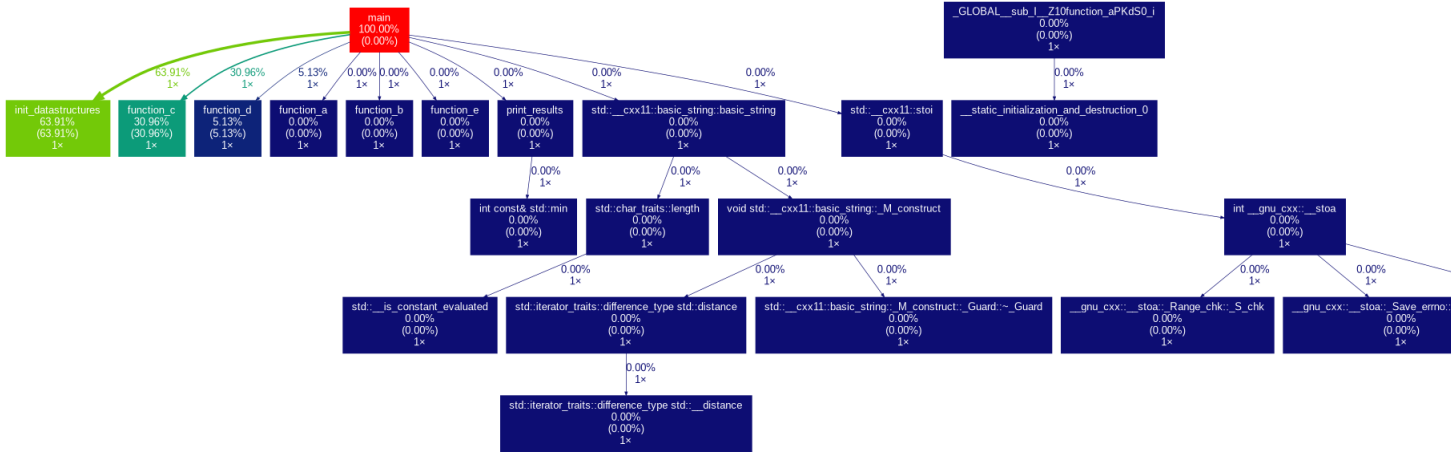


FIG. 1: Call Graph for $N = 40000$, at optimisation O1

The results from profiling using gprof agree with what is expected from the algorithmic analysis. It is worth noting that this profiling was performed on code compiled at O1 optimisation. From inspection of the code, functions a and b scale as $O(N)$, whereas functions c and d scale as $O(N^2)$. Therefore, for sufficiently large N , the runtime of the latter functions is expected to dominate, regardless of differences in operational intensity.

It is important to note that gprof uses a sampling-based profiling method with a sampling interval of approximately 0.01 seconds. Due to this relatively large interval, short-lived functions may not accumulate any samples by chance, even though they are executed. This explains why functions a , b , and e appear with a reported runtime of 0.00 seconds in the profile. This highlights the common issue and drawback with sampling based profilers.

The profile shows that, aside from the initialisation routine, functions c and d dominate the execution time. In particular, function c accounts for approximately 31.0% of the runtime, while function d

contributes around 5.1%. This is consistent with expectations, as part 1.1 shows function *c* performs significantly more work than function *d*, both in terms of floating-point operations and memory accesses under the assumed pessimistic memory model.

I have used gprof2dot in order to make the call graph image. This includes many of the helper and initialisation routines that are not important for this profiling.

C. 1.3

In order to benchmark the memory and floating point performance of the hotspot function (function *c*), the MEM_DP performance group from LIKWID was used to profile the code compiled with optimisation level O1. The results show a double precision floating point throughput of 606.5 MFLOP/s, a memory bandwidth of 2482.5 MB/s, and an operational intensity of 0.2443 FLOP/Byte. The observed CPI is 0.999, indicating relatively efficient instruction throughput with few pipeline stalls.

The theoretical peak floating point throughput of a Hamilton compute node is approximately 53.6 GFLOP/s. This is calculated assuming a boost clock frequency of 3.35 GHz, a vector width of four double-precision values (256-bit AVX), two floating point operations per fused multiply-add (FMA), and two FMA units:

$$3.35 \times 10^9 \times 4 \times 2 \times 2 = 53.6 \text{ GFLOP/s.}$$

The measured performance therefore achieves only around 1% of the theoretical peak. While this might initially suggest that the function is memory bound, this conclusion must be considered alongside the memory bandwidth results.

The measured peak memory streaming bandwidth on Hamilton (using the triad benchmark of likwid-bench) is approximately 24,000 MB/s for large working sets. The observed bandwidth of 2482.5 MB/s corresponds to only about 10% of this peak. This indicates that the code is not saturating the memory subsystem. Combined with the CPI being close to 1, this suggests that the processor is not stalled waiting on memory and that the function is not strongly memory bound.

As the results indicate the computation is neither compute nor memory bound it indicates the function does not fully utilise the compute units nor the memory bandwidth.

The measured operational intensity of 0.2443 FLOP/Byte is a factor of 2 higher than the estimate from Part 1.1. This discrepancy arises due to my estimate assuming every inner loop iteration contributes a store to *y[i]*. In reality the compiler optimises this away leaving only *N* stores within the outer loop. Including this change to the calculation of OI increases the estimated OI to $\frac{1}{4}$ for large *N*. This is then consistent with the measured operational intensity. ch

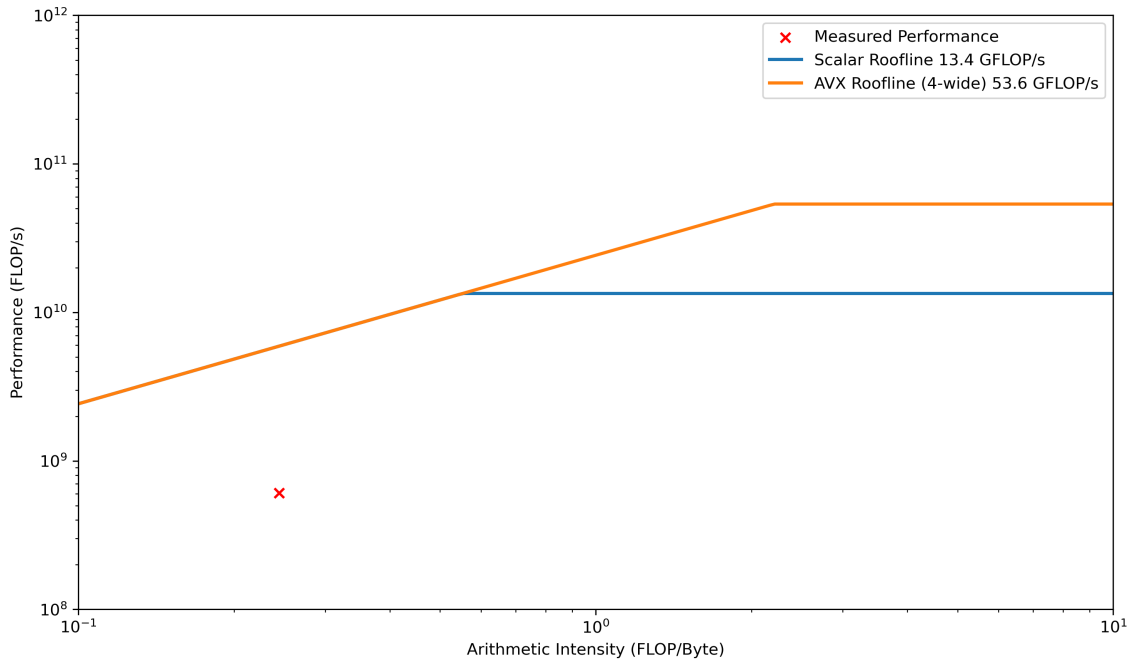


FIG. 2: Roofline plot for function c

D. 1.4

The Roofline model shows that the performance of function C sits significantly below the memory bandwidth ceiling, despite an operational intensity of 0.2443 FLOP/Byte. Combined with the observed memory bandwidth of only 2482.5 MB/s (approximately 10% of the peak), this indicates that the code is not memory-bound. This measured CPI is ≈ 1 , indicating the CPU is not heavily stalled and instructions are executed efficiently. Despite this smooth execution the achieved throughput is still significantly lower than the peak. Together these observations suggest the computational resources are underutilised. A relevant optimisation technique is therefore to increase vectorisation in order to improve floating point throughput. This can be achieved by restructuring the loop to enable and enforce SIMD execution, ensuring data alignment, and eliminating loop-carried dependencies that inhibit auto-vectorisation.

II. GPU PROGRAMMING

A. 2.2

See Figure 3

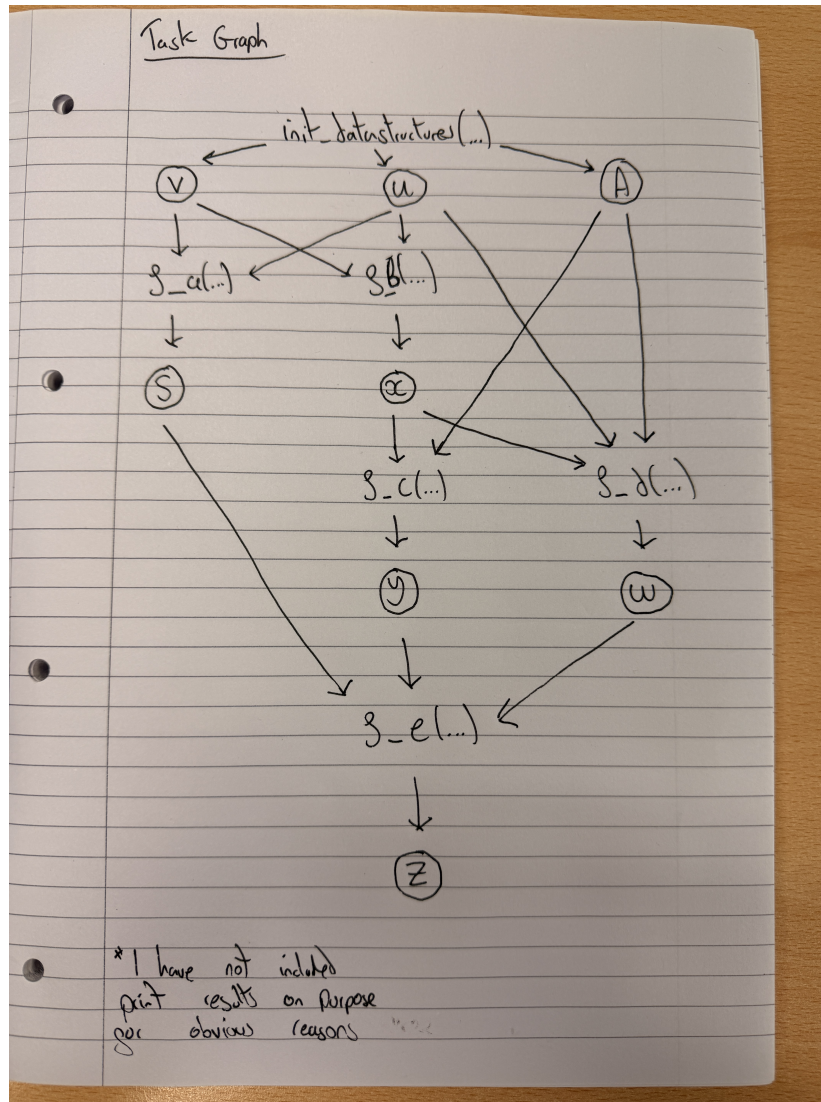


FIG. 3: Task graph for number_crunching.cpp

B. 2.4

CUDA has been used in order to port the source code for use on a GPU. As for memory management, Nvidia's CUDA documentation implies that asynchronous memory transfer between the host and device is only possible when host arrays are page-locked (pinned), i.e. declared using `cudaMallocHost()`. Memory transfers dominate the runtime of the entire program as is clearly visible in figures 5 and 6, with the large difference between CUDA total and CUDA kernel runtime being accounted for by the memory transfer of the $N \times N$ matrix A. Fortunately, matrix A and vectors u,v only need to be transferred once to the GPU after initialisation where they can remain until freed. The vectors returned from the other functions can then be transferred back to the host asynchronously upon their respective kernel returning.

CUDA Streams were used to enable concurrent execution of kernels and memory transfers. Kernels and transfers were queued in their respective streams utilising stream events to account for task

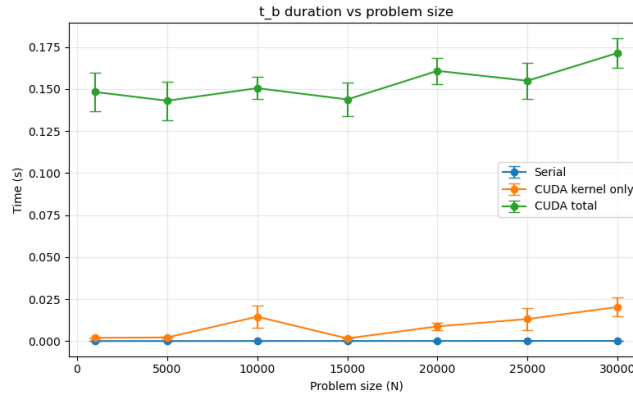


FIG. 4: Function b timings, distinguishing between the entire runtime for the function including memory transfer and the isolated kernel execution

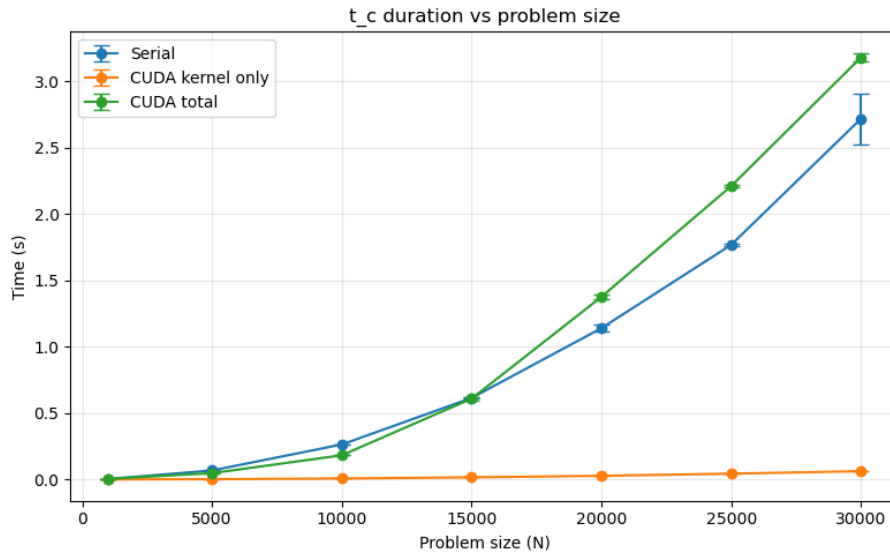


FIG. 5: Function c timings, distinguishing between the entire runtime for the function including memory transfer and the isolated kernel execution

dependencies. After sufficient kernels had been queued the serial CPU work of *function_a* can then be executed concurrently alongside the GPU work. At large N , the asynchronous execution resulted in a lower achieved runtimes, $T(N)$, (Figure 8) on the GPU port compared with the serial version, despite the individual ported functions often showing larger runtimes (Figures 4-7).

A naive, but effective, model of $T(N)$ is a quadratic fit. As the matrix A scales in size with N^2 it is not unreasonable to assume the memory transfers scale with $O(N^2)$. Furthermore we know from Section 1 that the number of loads, stores and flops for functions c and d scale with $O(N^2)$. A quadratic fit was applied (Figure 8) resulting in

$$T(N)_{serial} = 1.01e - 8N^2 - 1.05e - 5N + 2.38e - 2. \quad (1)$$

$$T(N)_{GPU} = 8.05e - 9N^2 + 7.54e - 6N + 1.97e - 1. \quad (2)$$

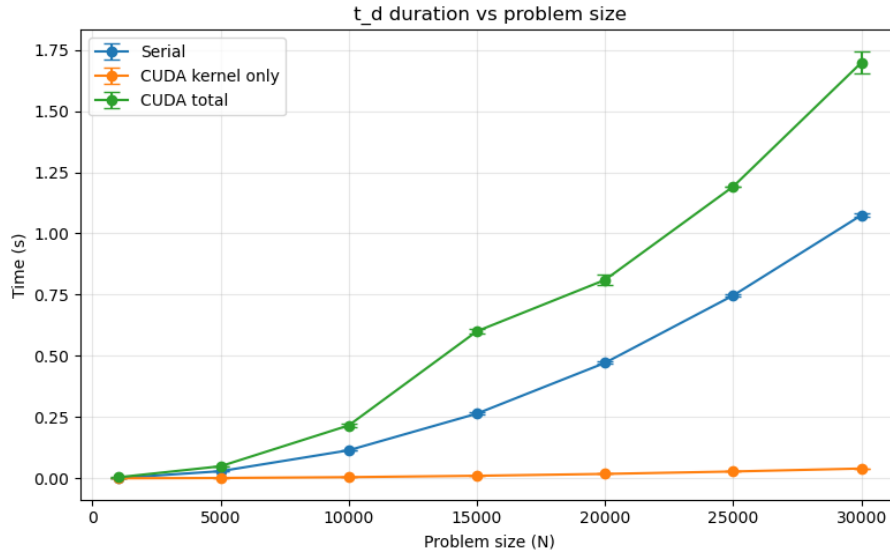


FIG. 6: Function d timings, distinguishing between the entire runtime for the function including memory transfer and the isolated kernel execution

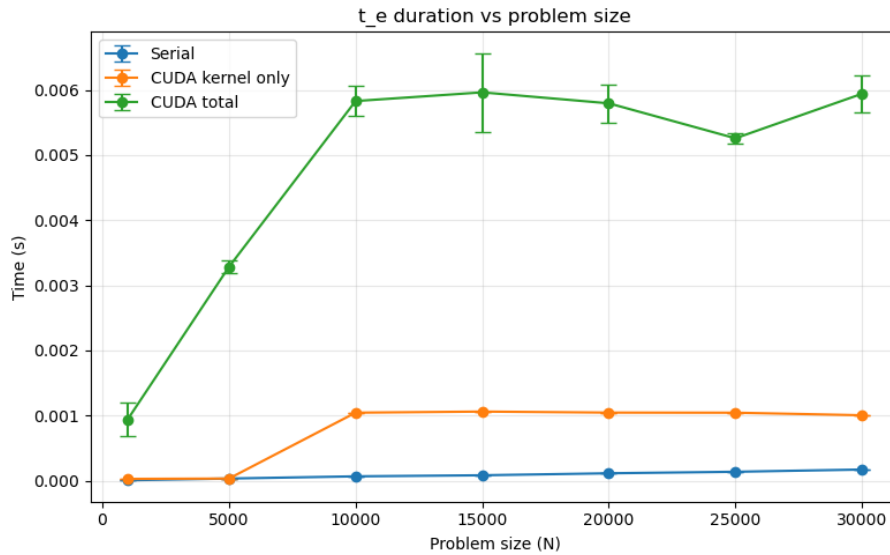


FIG. 7: Function d timings, distinguishing between the entire runtime for the function including memory transfer and the isolated kernel execution

. At large N $N > \approx 15000$, $T(N)_{GPU} < T(N)_{serial}$.

Upon successfully designing the codebase and producing correct outputs, Nvidia Nsight Compute (NCU) was used to profile the code. As expected, kernels c and d dominate the runtime. Despite both having theoretical occupancies of 100% the achieved occupancies are 49.7% and 49.8% respectively. Both of these kernels had consistently high device memory throughput (78.24%, 87.41%) indicating the memory bandwidth is the bottleneck and a possible cause of the lower occupancies. The grid and block sizes chosen were consistently a block size of 256 and grid size of $2 * (SM_count)$. These were chosen purposefully, with a multiple of 32 being chosen for the block size to be consistent with the warp size,

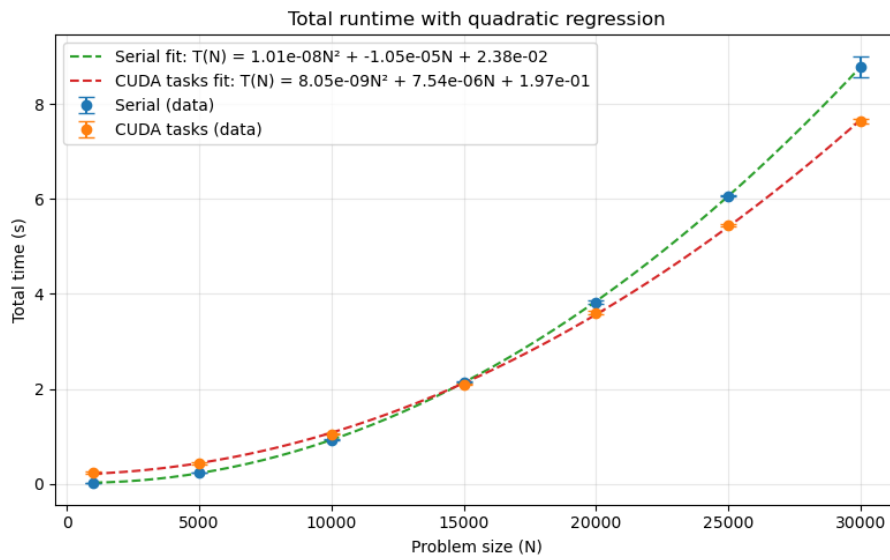


FIG. 8: Total runtime of serial program compared with *number_crunching_tasks.cu* with quadratic fit.

and a large enough grid to enable sufficient use of the GPUs resources. NCU did not indicate that these grid and block sizes were any cause of a bottleneck or reduced occupancy. Finding an "optimal" kernel launch configuration is often hardware dependent and difficult, thus I have deemed it appropriate to stick with my current static configuration.